Разработка страницы с формой регистрации.

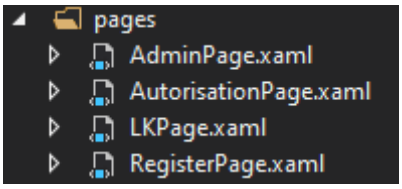
В файле MainWindow.xaml внутри тега <Grid> отобразим только один элемент <Frame>. Зададим ему имя и уберем встроенную панель навигации.

```
<Grid>
    <Frame Name="MainFrame" NavigationUIVisibility="Hidden"></Frame>
</Grid>
```

Для самой оконной формы зададим максимальную размерность с помощью свойства WindowsState.

```
WindowState="Maximized"
```

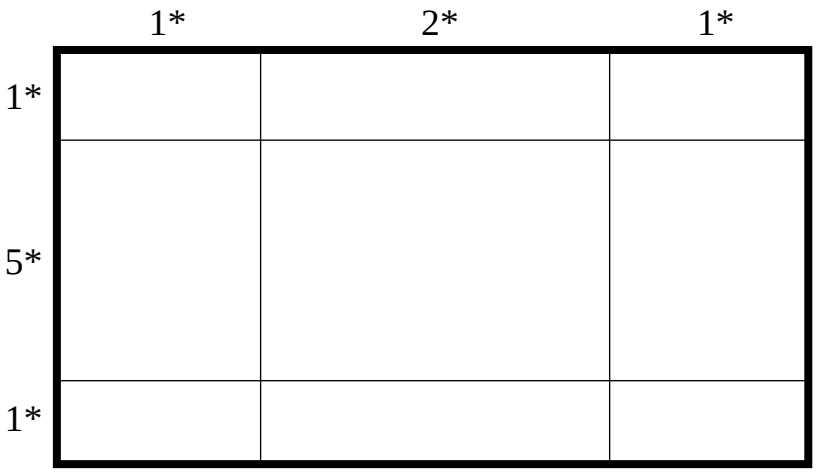
Создадим папку pages с будущим страницами приложения. Это страница с формой регистрации, страница с формой авторизации, личный кабинет пользователя и панель администратора.



При загрузке приложения в фрейм будем передавать страницу с формой регистрации.

```
public MainWindow()
{
    InitializeComponent();
    MainFrame.Navigate(new pages.RegisterPage());
}
```

На странице регистрации отобразим разметку.

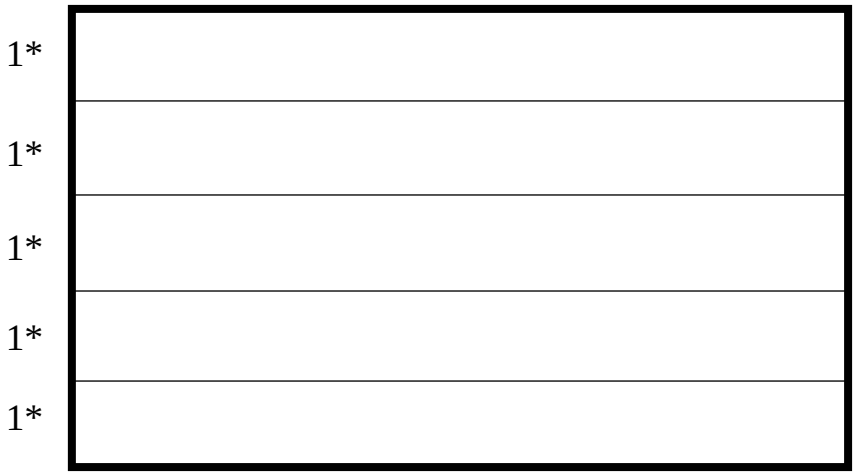


```
<Grid.RowDefinitions>
    <RowDefinition/>
    <RowDefinition Height="5*"/>
    <RowDefinition/>
</Grid.RowDefinitions>
<Grid.ColumnDefinitions>
    <ColumnDefinition/>
    <ColumnDefinition Width="2*"/>
    <ColumnDefinition/>
</Grid.ColumnDefinitions>
```

В центральной области разметке отобразим элемент <Grid> с границей <Border>. Ширина линии обводки 5, цвет черный и скругленные углы на 50.

```
<Border Grid.Row="1" Grid.Column="1" BorderThickness="5" BorderBrush="Black" CornerRadius="50">
    <Grid>
    </Grid>
</Border>
```

В разметке добавим 5 одинаковых строк.




```
<Grid.RowDefinitions>
    <RowDefinition/>
    <RowDefinition/>
    <RowDefinition/>
    <RowDefinition/>
    <RowDefinition/>
</Grid.RowDefinitions>
```

В первой строке отобразим заголовок «Регистрация» с размером текста 50.

```
<TextBlock TextAlignment="Center" VerticalAlignment="Center" FontSize="50">Регистрация</TextBlock>
```

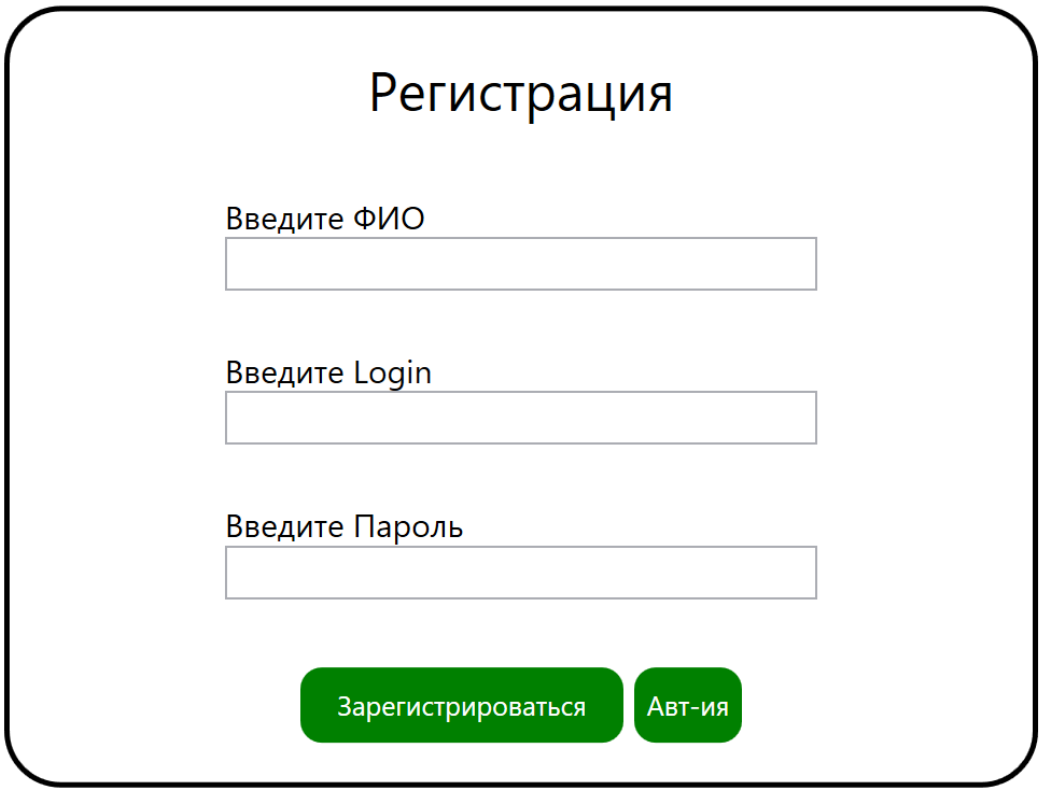
В следующих строках с помощью контейнера компоновки <StackPanel> добавим поля ввода регистрации. Поле для ФИО, поле для Логина, и поле для пароля. Связка с базой данных осуществляется с помощью метода Binding свойства Text.

```
<StackPanel Grid.Row="1" HorizontalAlignment="Center" VerticalAlignment="Center">
    <TextBlock FontSize="30">Введите ФИО</TextBlock>
    <TextBox Text="{Binding fio}" BorderThickness="2" Width="550" Height="50" Padding="10 2" FontSize="30"></TextBox>
</StackPanel>
```

В последней строке отобразим контейнер компоновки с двумя кнопками «Зарегистрироваться» и «Авторизация». Заливку и цвет обводки кнопкам сделать прозрачными с помощью параметра Transparent.

```
<StackPanel Grid.Row="4" Orientation="Horizontal" HorizontalAlignment="Center">
    <Border  Width="300" Height="70" CornerRadius="20" Background="Green" Margin="0 0 10 0">
        <Button Content="Зарегистрироваться" Background="Transparent" BorderBrush="Transparent" Foreground="White" FontSize="25" Click="Register_Click"/>
    </Border>
    <Border  Width="100" Height="70" CornerRadius="20" Background="Green">
        <Button Content="Авт-ия" Background="Transparent" BorderBrush="Transparent" Foreground="White" FontSize="25" Click="Auto_Click"/>
    </Border>
</StackPanel>
```

Интерфейс страницы с формой регистрации выглядит следующим образом.



На странице RegisterPage.xaml.cs создадим новый экземпляр сущности User, который будет являться текущим пользователем.

```
private User _currentUser = new User();
```

При инициализации странице добавим к контексту данных этого текущего пользователя.

```
DataContext = _currentUser;
```

Добавим обработчики событий у кнопок.

При нажатии на кнопку «Авторизация» осуществляется переход на страницу AutorisationPage.

```
NavigationService.Navigate(new AutorisationPage());
```

При сохранении добавим проверки валидаций на наличие и корректное заполнение данных. С помощью объекта типа StringBuilder создаем список из следующих ошибок.

```
StringBuilder errors = new StringBuilder();

if (string.IsNullOrWhiteSpace(_currentUser.fio))
    errors.AppendLine("Укажите ФИО");
if (string.IsNullOrWhiteSpace(_currentUser.login))
    errors.AppendLine("Укажите Login");
if (string.IsNullOrWhiteSpace(_currentUser.password))
    errors.AppendLine("Укажите пароль");
```

Если длина списка с ошибками превышает 0, то производим вывод этих ошибок.

```
if (errors.Length > 0)
{
    MessageBox.Show(errors.ToString());
    return;
}
```

Если же все хорошо, и у нас происходит операция добавления (т.е. еще не присвоен код нового отеля), то мы будем пытаться добавить модель или экземпляр созданного отеля. Получив его контекст и обратившись к таблице отелей, с помощью метода Add мы добавляем созданный экземпляр.

Напишем код для сохранения изменений, используя метод SaveChanges. Этот метод является коварным, его необходимо поместить в блок try-catch, чтобы он отработал корректно, и в случае возникновения какой-либо непредвиденной ошибки, приложение не «упало», а корректно работало. Мы выведем сообщение об ошибке, если она появилась. В случае успешного сохранения выведем сообщение о том, что информация сохранена.

```
if (_currentUser.id == 0)
{
DB.Context.User.Add(_currentUser);
}

try
{
    DB.Context.SaveChanges();
    MessageBox.Show("Информация сохранена", "Сообщение", MessageBoxButton.OK, MessageBoxImage.Information);

}
catch (Exception ex)
{
    MessageBox.Show(ex.Message.ToString(), "Сообщение", MessageBoxButton.OK, MessageBoxImage.Information);
}
```

Также при регистрации новых пользователей их роль администратора по умолчанию ложна.

```
_currentUser.admin = false;
```

Проверьте работу регистрации, создав пользователя: Фамилия Имя Отчество – Логин: fio – Пароль: 123;